PHASE-II: PROJECT AND TEAM INFORMATION

Project Title

ShelfSense – Intelligent Library Resource and Reservation System.

Student / Team Information

Team: [Team Name] ID: DSCPP-III-2026-T306		*Photo*
Team member 1 (Team Lead)	*Sarthak Pandey – 2510380166* *2510380166@geu.ac.in* *University Roll No.: 2028487* *Phone No.: 7897680580*	
Team member 2	*Anurag Chauhan – 2510370177* *2510370177@geu.ac.in* *University Roll No.: 2028652* *Phone No.: 6398498642*	
Team member 3	*Ashutosh Singh – 2510379890* *2510379890@geu.ac.in* *University Roll No.: 2028675* *Phone No.: 7366001531*	
Team member 4	*Pankaj Kumar – 2512510018* *2512510018@geu.ac.in* *University Roll No.: 2028789* *Phone No.: 6205007902*	

IMPLEMENTATION AND PROGRESS

Project Summary and Phase-II Objective

ShelfSense solves an everyday library problem: who gets a popular book next, and where is it kept? Phase I covered planning and design; Phase II builds the working core backend (C for DSA, C++ for OOP). Objectives:

O1. *Build the console core backend.* ***O2.*** *Manage books and students.* ***O3.*** *Issue and return books.*
O4. *FIFO and priority reservations.* ***O5.*** *Efficient searching and sorting.* ***O6.*** *Transaction history (stack).*
O7. *Library navigation (graphs).* ***O8.*** *Data persistence (files).*

Work Completed Since Phase I

The console program covers book management (add, display, search, update, delete and check availability, with several copies per book), student management (add, display and search by ID or name), and issue and return with a fixed limit of books per student. If every copy is out, the student can reserve the book. When a copy comes back, it is issued automatically to the next eligible student, availability is updated and the transaction is recorded. All data is saved in text files and loaded again at start-up. To be clear about what is finished: this is a console backend only. A GUI, a web screen and a real database belong to Phase III.

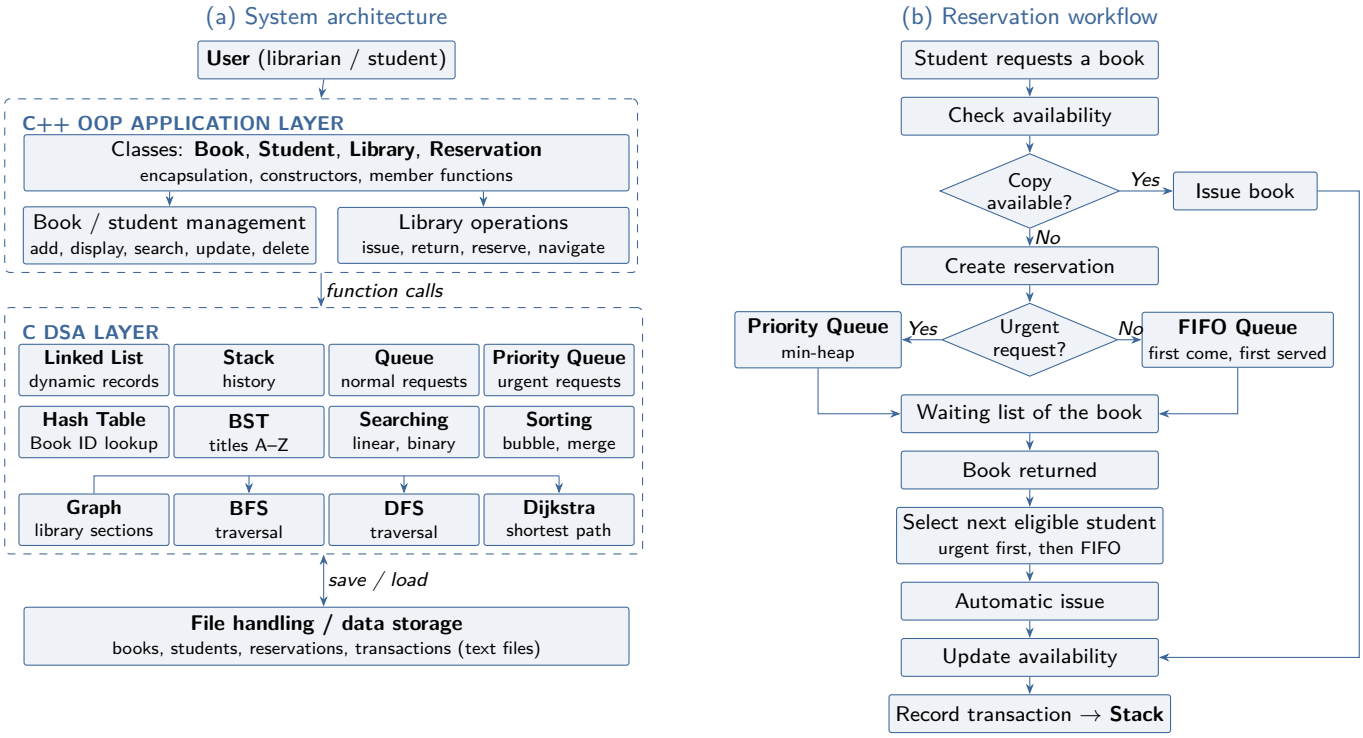
Data Structures, Algorithms and Class Design

C DSA layer. *A linked list stores records dynamically (insert, delete, traverse, search). Linear search finds books by keyword and students by ID or name; binary search finds a book by ID in the sorted list. Bubble sort orders books by ID and merge sort orders them by title. A hash table with chaining gives fast Book ID lookup, and a binary search tree keeps titles in order, so an inorder traversal lists them A–Z.*

Reservations, history and map. *Normal requests wait in a FIFO queue. Urgent requests go to a priority queue built as a min-heap: the student who needs the book in the fewest days goes first, and equal requests keep their request order. A stack records every ISSUE, RETURN, RESERVE and AUTO-ISSUE, newest on top. Library sections form a weighted graph; BFS and DFS traverse it and Dijkstra finds the shortest walking path.*

C++ classes. *Book, Student, Library and Reservation keep their data private behind public member functions, constructors and getters/setters (encapsulation). The Library class calls the C modules, so the OOP layer stays separate from the DSA layer. Inheritance and polymorphism are not forced; we will add them only where they genuinely fit.*

System Architecture and Reservation Workflow



(a) *The C++ classes take the user's request and call the C DSA modules, which read and write the data files.* ***(b)*** *A free copy is issued at once; otherwise the request waits in the priority queue (urgent) or the FIFO queue (normal). On return, urgent requests are served first, then FIFO, and the copy is issued automatically. A book that is not in the catalogue is never issued or reserved, and available copies never go below zero.*

EVALUATION, CONTRIBUTIONS AND NEXT STEPS

Initial Evaluation and Test Plan

At this stage we check that every operation behaves correctly and the same way each time, using a small fixed set of sample books, students and library sections. We are not claiming that ShelfSense shortens waiting time in a real library. The table below is our test checklist: for each case we compare the console output with the expected result and note any mismatch.

Case	What should happen
1. Add a book	Book is stored with its copies; a duplicate Book ID is refused.
2. Search existing book	The correct record is shown, with total and available copies.
3. Search missing book	A clear "not found" message is shown and nothing is changed.
4. Issue available book	Book is issued, available copies go down by one and an ISSUE record is pushed on the stack.
5. Return a book	Available copies go up by one. Returning a book the student does not hold is refused.
6. Reserve unavailable book	With all copies out, the student joins that book's waiting list; a duplicate reservation is refused.
7. FIFO reservation	Normal requests are served in the order they were made.
8. Priority reservation	An urgent request is served before normal ones; equal urgency keeps request order.
9. Stack history	The latest transaction is on top; ISSUE, RETURN, RESERVE and AUTO-ISSUE are all recorded.
10. Hash table search	Lookup by Book ID gives the same record as linear search, also for IDs in the same bucket.
11. BST search / traversal	Title search finds the book; inorder traversal lists the titles A–Z.
12. BFS / DFS	Every library section is visited exactly once from the chosen start section.
13. File save / load	Data saved before exit loads back unchanged; invalid lines are skipped without a crash.

PROBLEM FACED *When the program ended or the user logged out, all data was reset; books, students and reservations were lost.*

OUR SOLUTION ***File handling****: data is saved to files before the program ends and loaded again at start-up, so it persists.*

Individual Contributions

This is the planned division of the Phase-II work. It shows who owns which part.

Sarthak Pandey: *Architecture, reservation logic, integration and documentation.*

Anurag Chauhan: *Linked list, queue, priority queue and stack modules.*

Ashutosh Singh: *Hash table, BST, graph module, BFS, DFS and Dijkstra.*

Pankaj Kumar: *C++ OOP classes, file handling, testing and report/documentation.*

Limitations and Remaining Work

ShelfSense is a console-based backend. Data is kept in text files instead of a database, and the files are not safe for many users at the same time. There is no GUI or web interface and no QR/barcode scanning yet. Library distances, borrowing limits and the reservation priority rule are values and rules we defined for the project, not a library standard, and the binary search tree is not self-balancing. We still need to test with larger data sets and run FIFO and priority reservation side by side, with the same requests, before we say anything about waiting time.

Next Milestones and Phase-II Deliverables

Phase-II deliverables: *The ShelfSense source code (C DSA modules and C++ classes), sample data files, the test checklist above and this report. Before the next review we will finish the remaining test cases and record the results we observe.*

Planned for Phase III (not part of Phase II): *A GUI or web interface, database integration, QR/barcode scanning, notifications, a simple book recommendation system, advanced search, and automatic due dates and fines.*

References

1. Cormen et al., Introduction to Algorithms, 4th ed., MIT Press, 2022: sorting, heaps, hashing, BSTs, BFS, DFS and shortest paths.

2. E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, 1, 269–271, 1959.

3. Kernighan and Ritchie, The C Programming Language, 2nd ed., 1988; Horowitz, Sahni and Anderson-Freed, Fundamentals of Data Structures in C, 2nd ed., 2008.

4. B. Stroustrup, The C++ Programming Language, 4th ed., 2013: classes, constructors and encapsulation.

5. S. R. Ranganathan, The Five Laws of Library Science, 1931. Used for background only; ShelfSense is not a real library management system.